



Tecnológico de Monterrey

Curso:

Modelación de sistemas multiagentes con gráficas computacionales

Grupo:

302

Estudiantes:

Santiago Arista Viramontes - A01028372

Fausto Izquierdo Véjar - A01785221

Maestros:

Octavio Navarro Hinojosa

Gilberto Echeverria

Título:

Reto: Movilidad Urbana

Fecha de entrega:

04 de Diciembre del 2025

Reporte Técnico: Sistema Multi-Agentes para Movilidad Urbana

Simulación de Tráfico con Mariposas y Árboles

1. Problema y Propuesta de Solución

1.1 Problema

La movilidad urbana en México enfrenta una crisis significativa. El crecimiento indiscriminado del uso de automóviles ha generado:

- **Congestión vehicular masiva:** Los Kilómetros-Auto Recorridos (VKT) se han triplicado de 106 millones en 1990 a 339 millones en 2010.
- **Impactos negativos:** Contaminación (smog), accidentes viales, enfermedades respiratorias y pérdidas económicas por tiempo perdido en tráfico.
- **Ineficiencia del sistema:** Falta de coordinación entre vehículos, semáforos mal sincronizados y ausencia de rutas optimizadas.

El problema fundamental es la falta de inteligencia distribuida en el sistema de tráfico, donde cada vehículo actúa de manera independiente sin considerar el estado global del sistema, generando patrones de congestión predecibles y evitables.

1.2 Propuesta de Solución

Se desarrolló un Sistema Multi-Agentes (SMA) que simula el tráfico urbano utilizando:

1. Agentes autónomos inteligentes (Mariposas/Vehículos):

Cada mariposa es un agente con capacidad de percepción del ambiente (4 celdas de visión [adelante, atrás, derecha e izquierda]) que utiliza el algoritmo de pathfinding A* para encontrar rutas óptimas

Cada mariposa utiliza lógica de evitación de congestión y rerouting dinámico

mientras respeta semáforos y obstáculos.

2. Ambiente simulado:

Ciudad con calles direccionales (28x28 celdas [variable]), semáforos con sincronización temporal, obstáculos (árboles/edificios) y puntos de origen y destino

3. Optimización de throughput:

Algoritmos diseñados para minimizar el tiempo promedio de viaje, evitar congestión mediante rerouting inteligente, distribución del tráfico uniformemente en la red vial y maximización del número de vehículos que completan su trayecto.

4. Visualización 3D:

Representación gráfica en WebGL con modelos 3D temáticos (mariposas como vehículos, árboles y rocas como edificios), iluminación dinámica para mostrar el estado de semáforos, transformaciones matriciales para movimiento realista

2. Diseño de los Agentes

2.1 Agente Car (Mariposa/Vehículo)

Tiene como objetivo desplazarse desde un punto de origen (esquina del mapa) hasta un destino asignado en el menor tiempo posible, evitando colisiones y respetando las reglas de tránsito.

Capacidad Efectora

El agente puede realizar movimiento direccional, avanzando hacia adelante según el camino y haciendo cambios de carril diagonales (forward + left/right), aunque no puede desplazarse lateralmente sin avanzar ni retroceder. También cuenta con pathfinding dinámico, utilizando A* para calcular la ruta óptima, recalculándola cuando detecta obstáculos o congestión, y ajustando su estrategia con base en el tráfico en tiempo real. Finalmente,

incorpora control de estado, deteniéndose ante semáforos en rojo, esperando cuando la celda siguiente está bloqueada y auto-eliminándose al llegar a su destino.

Percepción del Ambiente

El agente percibe su entorno mediante el método ``get visible cells()``:

```
visible_cells = {  
    'ahead_1': celda 1 posición adelante,  
    'ahead_2': celda 2 posiciones adelante,  
    'left': celda lateral izquierda,  
    'right': celda lateral derecha  
}
```

La información que recibe el agente por celda incluye la presencia de otros vehículos, la detección de obstáculos físicos como árboles o edificios, el estado de los semáforos con su color y tiempo de cambio, el tipo de camino junto con su dirección, y también la ubicación de los destinos; todo esto dentro de un rango de percepción de 5 celdas que aporta una visión limitada y realista.

Proactividad

El agente Car es altamente proactivo:

Planificación anticipada:

Calcula la ruta completa al inicio mediante `update_path()` y reevalúa constantemente las condiciones del entorno para anticiparse a posibles problemas.

Toma de decisiones autónoma:

Evita congestión proactivamente

```
if car_count_next >= 2:

    if self.stuck_counter >= 2:

        self.update_path() # Busca ruta alternativa
```

En esta etapa, el agente determina por sí mismo cuándo evitar zonas congestionadas y cuándo modificar su trayectoria sin depender de instrucciones externas.

Adaptación dinámica:

Detecta bloqueos utilizando *stuck_counter*, recalcula su ruta si permanece atascado durante tres o más pasos y evita entrar en áreas con alta congestión antes de alcanzarlas.

Optimización de semáforos:

```
if time_to_change <= 2:

    move_cost += 1 # Espera si cambiará pronto

else:

    move_cost += time_to_change * 0.5 # Evita si tarda mucho
```

El agente ajusta su comportamiento según los semáforos, priorizando movimientos eficientes y minimizando tiempos de espera.

Métricas de Desempeño

El sistema implementa diversas métricas de evaluación que permiten analizar el rendimiento del tráfico simulado. Total Spawned mide el número total de vehículos generados en la simulación mediante *total_cars_spawned*. Currently Active contabiliza los vehículos que siguen activos utilizando *get_current_car_count()*. Reached Destination registra cuántos vehículos lograron completar su trayecto con *total_cars_reached_destination*. La métrica Throughput Rate indica la eficiencia del sistema calculando la proporción de vehículos completados por step mediante *reached / total_steps*.

Congestion Level identifica el grado de congestión contando los vehículos que permanecen atascados, definidos como aquellos con *stuck_counter* > 3. Finalmente, Failed Spawn Rate mide la cantidad de intentos fallidos al generar nuevos vehículos utilizando *failed_spawn_attempts*.

2.2 Agente Traffic_Light (Semáforo)

Regula el flujo de tráfico mediante cambios de estado temporizados para evitar colisiones en intersecciones.

Capacidad Efectora

Cambiar estado: *False* (rojo) ↔ *True* (verde). Cada *timeToChange* steps.

Percepción

No percibe el ambiente (agente reactivo simple), solo mantiene estado interno (contador de tiempo)

Proactividad

No es proactivo, es un agente reactivo temporal ya que cambia estado basándose únicamente en contador interno. Hay dos tipos: cambio cada 7 steps ('s') y cada 15 steps ('S')

2.3 Agentes Estáticos

Obstacle (Árbol/Edificio)

- Objetivo: Bloquear ciertas celdas del mapa
- Capacidad efectora: Ninguna (estático)
- Percepción: Ninguna

- Proactividad: No aplica

Road (Camino)

- Objetivo: Definir dirección de flujo vehicular
- Capacidad efectora: Ninguna (información pasiva)
- Percepción: Ninguna
- Proactividad: No aplica
- Información: Dirección (Up, Down, Left, Right)

Destination (Destino)

- Objetivo: Marcar puntos de llegada
- Capacidad efectora: Ninguna
- Percepción: Ninguna
- Proactividad: No aplica

3. Arquitectura de Subsunción de los Agentes

La arquitectura de subsunción es un modelo de control por capas donde comportamientos de bajo nivel tienen prioridad sobre comportamientos de alto nivel. En este sistema, el agente Car (mariposa) implementa una arquitectura de subsunción con las siguientes capas:

3.1 Jerarquía de Comportamientos (de menor a mayor nivel)

CAPA 0: SUPERVIVENCIA - ALCANZAR DESTINO



CAPA 1: PLANIFICACIÓN - SEGUIR RUTA A*



CAPA 2: ADAPTACIÓN - RE-PLANNING DINÁMICO



CAPA 3: EVITAR CONGESTIÓN Y ESPERAS LARGAS



CAPA 4: RESPETAR SEMÁFOROS



CAPA 5: EVITAR COLISIONES

3.2 Implementación Detallada

CAPA 0: SUPERVIVENCIA - Alcanzar Destino [MENOR PRIORIDAD]

Función: Objetivo final del agente (razón de existir)

Implementación:

En step() - Primera línea del método

```
if self.destination and self.cell == self.destination.cell:
```

```
    self.reached_destination = True
```

```
    self.model.increment_cars_reached_destination()
```

```
# Auto-eliminación
```

```
if self in self.cell.agents:
```

```
    self.cell.agents.remove(self)
```

```
if self in self.model.agents:
```



```
self.model.agents.remove(self)

return # ÉXITO - Terminar existencia
```

Características:

- **Meta final:** Razón de ser del agente
- **No bloquea:** Se ejecuta solo al llegar
- **Termina vida:** Auto-destrucción al completar objetivo
- **Menor prioridad ejecutiva:** Todas las capas pueden impedirla

CAPA 1: PLANIFICACIÓN - Seguir Ruta A* [PRIORIDAD BAJA]

Función: Ejecutar plan de ruta óptima pre-calculado

Implementación:

```
# En step() - Verificación inicial de path

if not self.path:

    self.update_path() # Calcular ruta si no existe

    if not self.path:

        return # No hay ruta posible


# Obtener siguiente celda del plan

next_cell = self.get_next_cell_from_path()

if next_cell is None:

    return # Fin del plan
```

Algoritmo de planificación:

```
def a_star_pathfinding(self):

    # Heurística: distancia Manhattan + penalizaciones
```

```
f_score = g_score + h_score
```

```
# Penalizaciones incluidas en g_score:
```

```
# - Movimiento diagonal: +1.4
```

```
# - Movimiento recto: +1.0
```

```
# - Congestión: +4 por cada carro
```

```
# - Semáforo rojo: +0.5 * time_to_change
```

Características:

- **Comportamiento deliberativo:** Planifica antes de actuar
- **Búsqueda óptima:** Minimiza costo con A*
- **Considera entorno:** Penaliza congestión y semáforos
- **Puede ser inhibida:** Por TODAS las capas superiores

CAPA 2: ADAPTACIÓN - RE-PLANNING DINÁMICO [PRIORIDAD MEDIA-BAJA]

Función: Recalcular ruta cuando el plan falla

Implementación:

```
# En step() - Cuarta verificación (dentro de is_cell_blocked)
```

```
if self.is_cell_blocked(next_cell):
```

```
    has_obstacle = any(isinstance(agent, Obstacle) for agent in next_cell.agents)
```

```
    if has_obstacle:
```

```
        self.update_path() # SUBSUME CAPA 1 (ruta actual inválida)
```

```
    else:
```

```
        # Bloqueo temporal por otro carro
```

```
        if self.last_cell == self.cell:
```

```
            self.stuck_counter += 1
```

```

else:

    self.stuck_counter = 0

if self.stuck_counter >= 3:

    self.update_path() # SUBSUME CAPA 1 (atascado prolongado)

    self.stuck_counter = 0

```

Condiciones de activación:

- Obstáculo nuevo detectado en ruta
- Stuck_counter ≥ 3 (atascado 3+ turnos)
- Ruta actual no en path (ValueError)

Características:

- **Recuperación de fallos:** Repara planes rotos
- **Detección de bloqueo:** Identifica atascos prolongados
- **Memoria temporal:** Usa ``stuck_counter`` y ``last_cell``
- **Puede ser inhibida:** Por CAPA 3, 4 y 5

CAPA 3: EVITACIÓN - CONGESTIÓN Y ESPERAS LARGAS [PRIORIDAD MEDIA]

Función: Evitar áreas congestionadas proactivamente

Implementación:

En step() - Tercera verificación

```

car_count_next = sum(1 for agent in next_cell.agents
                      if isinstance(agent, Car) and not getattr(agent, 'reached_destination', False))

```

```

if car_count_next >= 2:

```

```
# SUBSUME CAPAS 1-2 (planificación y adaptación)
```

```
if self.stuck_counter >= 2:
```

```
    self.update_path() # Busca ruta con menos tráfico
```

```
    self.stuck_counter = 0
```

```
else:
```

```
    self.stuck_counter += 1
```

```
    self.last_cell = self.cell
```

```
    return # ESPERA un turno
```

Características:

- **Comportamiento anticipatorio:** Evita congestión antes de entrar
- **Contador de paciencia:** Espera 2 turnos antes de re-rutear
- **Optimización de throughput:** Distribuye tráfico
- **Puede ser inhibida:** Por CAPA 4 y 5

CAPA 4: CUMPLIMIENTO - RESPETAR SEMÁFOROS [MUY ALTA PRIORIDAD]

Función: Cumplir reglas de tránsito obligatorias

Implementación:

```
# En step() - Segunda verificación
```

```
is_red, time_to_change = self.is_traffic_light_red(next_cell)
```

```
if is_red:
```

```
    # SUBSUME CAPAS 0-3 (planificación, adaptación, etc.)
```

```
    if time_to_change > 8: # Semáforo recién cambió a rojo
```

```
        if self.stuck_counter >= 5:
```

```
            self.update_path() # Busca ruta alternativa
```

```
            self.stuck_counter = 0
```

```
return # NO SE MUEVE - PARADA OBLIGATORIA
```

Condiciones de activación:

```
def is_traffic_light_red(self, cell):  
  
    for agent in cell.agents:  
  
        if isinstance(agent, Traffic_Light):  
  
            if agent.state == False: # False = Rojo  
  
                return (True, agent.timeToChange)  
  
    return (False, 0)
```

Características:

- **Prioridad muy alta:** Solo superada por CAPA 5
- **Lógica con inteligencia:** Evalúa tiempo de espera
- **Excepción inteligente:** Si espera >5 steps, busca otra ruta
- **Legal:** Respetar leyes de tránsito

CAPA 5: SEGURIDAD - EVITAR COLISIONES [MÁXIMA PRIORIDAD]

Función: Prevenir choques físicos (supervivencia inmediata)

Implementación:

```
# En step() - Primera verificación crítica  
  
if self.is_cell_blocked(next_cell):  
  
    # SUBSUME TODAS LAS DEMÁS CAPAS  
  
    # No importa la ruta, semáforo, o plan  
  
    return # NO SE MUEVE - PARADA ABSOLUTA
```

Condiciones de activación:

```
def is_cell_blocked(self, cell):  
  
    for agent in cell.agents:  
  
        if isinstance(agent, Obstacle):  
  
            return True # OBSTÁCULO FÍSICO  
  
        if isinstance(agent, Car):  
  
            if not getattr(agent, 'reached_destination', False):  
  
                return True # OTRO VEHÍCULO  
  
    return False
```

Características:

- **Prioridad absoluta:** Anula todo lo demás
- **Respuesta instantánea:** Sin deliberación
- **Crítica de seguridad:** Evita choques mortales
- **Sin excepción:** Ninguna capa puede anularla

4. Características del ambiente

El ambiente en el que operan los agentes Car (mariposas) se clasifica según las siguientes dimensiones:

4.1. Accesibilidad: Parcialmente Observable

El ambiente es parcialmente observable (o inaccesible) porque:

- Los agentes solo pueden percibir 4 celdas alrededor de su posición actual (2 adelante, izquierda y derecha)
- No tienen acceso al estado global del sistema (posición de todos los vehículos, estado de todos los semáforos)
- No pueden "ver a través" de obstáculos o más allá de su rango de percepción limitado
- La información sobre congestión en otras partes del mapa no está disponible

Implicación: Los agentes deben tomar decisiones con información incompleta, lo que requiere estrategias reactivas y adaptativas.

4.2. Determinismo: No Determinista (Estocástico)

El ambiente es no determinista porque:

- Las decisiones de otros agentes son impredecibles (cada mariposa decide independientemente)
- Múltiples agentes pueden intentar moverse simultáneamente, creando situaciones impredecibles
- Aunque las reglas son deterministas, la interacción entre agentes genera resultados no predecibles

Implicación: No se puede predecir con certeza el estado futuro del sistema, requiere planificación adaptativa con re-planning frecuente.

4.3. Episodicidad: Secuencial (No Episódico)

El ambiente es secuencial porque:

- Las decisiones presentes afectan estados futuros: Elegir una ruta congestionada ahora resultará en retrasos después
- La posición actual determina opciones futuras: No se puede "deshacer" un movimiento
- El stuck_counter acumula información temporal que afecta decisiones futuras
- Los semáforos cambian con el tiempo, y esperar o no esperar afecta el plan completo
- La historia de movimientos (last_cell, path_index) influye en el comportamiento actual

Implicación: Los agentes deben considerar las consecuencias a largo plazo de sus acciones, no solo el beneficio inmediato.

4.4. Dinamismo: Dinámico

El ambiente es dinámico porque:

- El estado cambia mientras el agente delibera: Otros vehículos se mueven, semáforos cambian
- Los semáforos alternan entre rojo y verde cada N steps automáticamente
- Nuevos vehículos aparecen cada 10 steps en las esquinas
- Vehículos desaparecen cuando llegan a su destino
- La congestión fluctúa constantemente según el movimiento de otros agentes

Implicación: Los agentes no pueden asumir que el ambiente permanece estático durante la planificación, deben re-evaluar constantemente

4.5. Discretización: Discreto

El ambiente es discreto porque:

- El tiempo avanza en steps enteros, no de forma continua
- Las posiciones son coordenadas enteras (x, y), no valores reales
- Los estados de semáforos son binarios (rojo/verde, True/False)
- Las acciones son discretas (moverse a celda adyacente o quedarse)

Implicación: Simplifica el razonamiento y la planificación, permite algoritmos de búsqueda en grafos como A*.

5. Conclusiones

5.1 Logros del Sistema

1. Implementación exitosa de SMA inteligente: Se desarrolló un sistema donde cada agente toma decisiones autónomas basadas en percepción local, logrando comportamiento emergente global.

2. Optimización de throughput: El algoritmo A* con penalizaciones por congestión logra distribuir el tráfico eficientemente por medio de rerouting dinámico reduce tiempo de viaje promedio, detección de atascos permite reacción temprana y penalizaciones agresivas (4x por vehículo) previenen acumulación.

3. Arquitectura de subsunción robusta: La jerarquía de comportamientos garantiza seguridad, al evitar colisiones siempre de manera prioritaria, respeto a reglas de tránsito (semáforos) y optimización inteligente cuando es seguro.

4. Visualización 3D efectiva: La representación con mariposas y árboles proporciona una interfaz intuitiva y visualmente atractiva, retroalimentación en tiempo real del estado del sistema, e iluminación dinámica para identificar semáforos.

5.2 Limitaciones Identificadas

1. Percepción limitada:

Visión de solo 4 celdas puede causar decisiones subóptimas y no hay comunicación entre agentes (no cooperan activamente)

2. Semáforos no adaptativos:

Temporizadores fijos independientes del tráfico que no ajustan duración según congestión

3. Spawn en esquinas únicamente:

Limita realismo de entradas al sistema y puede generar congestión en puntos de origen

4. Escalabilidad computacional:

A* se ejecuta frecuentemente (cada *stuck_counter*) y un grid 28x28 es pequeño para ciudades reales